INFORMATIONSBLATT

IB01 – Sessions: Der Server merkt sich etwas

Dynamische Webseiten mit PHP · LS10 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

🎯 Darum geht es

Sie verstehen, warum sich ein Webserver von sich aus nichts merkt, speichern Daten seitenübergreifend in einer Session und leiten mit `header()` auf eine andere Seite weiter.

1 | HTTP vergisst alles

Jeder Seitenaufruf ist für den Server eine völlig neue Anfrage. Er weiß nicht, dass derselbe Browser vor zehn Sekunden schon einmal da war – HTTP ist **zustandslos**. Variablen verschwinden nach jedem Aufruf. Für eine Anmeldung braucht man aber ein Gedächtnis über mehrere Seiten hinweg.

2 | Die Session

Eine **Session** (Sitzung) ist ein Speicherbereich auf dem Server, der einem Browser zugeordnet ist. Beim ersten `session_start()` erzeugt PHP eine zufällige **Session-ID** und schickt sie als Cookie `PHPSESSID` an den Browser. Bei jedem weiteren Aufruf schickt der Browser das Cookie mit, und PHP holt die passenden Daten in das Array `$_SESSION`.

Befehl	Wirkung
`session_start();`	Session beginnen oder fortsetzen – auf jeder Seite, die `$_SESSION` braucht
`$_SESSION['name'] = 'Lena';`	Wert in der Session speichern
`$_SESSION['name']`	Wert lesen – auf jeder Seite derselben Sitzung
`unset($_SESSION['name']);`	einen Eintrag löschen
`$_SESSION = []; session_destroy();`	alles löschen und die Session beenden (Logout)

zaehler.php zaehler.php

```php
<?php
  session_start();                    // immer ganz am Anfang!

  if (!isset($_SESSION['besuche'])) {
    $_SESSION['besuche'] = 0;
  }
  $_SESSION['besuche']++;
?>
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Zähler</title></head>
<body>
  <p>Sie haben diese Seite in dieser Sitzung <?php echo $_SESSION['besuche']; ?>-mal aufgerufen.</p>
</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS

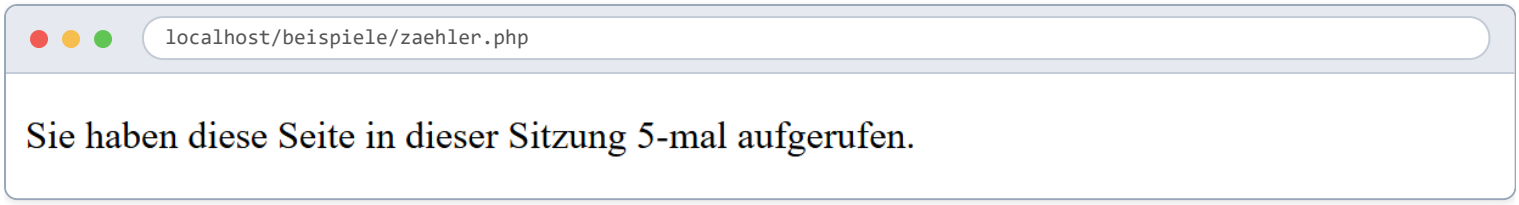


Abbildung 1: In dieser Sitzung wurde die Seite bereits viermal aufgerufen; jetzt zählt sie den fünften Besuch.

Seitenübergreifend: Seite A speichert, Seite B liest.

seite_a.php seite_a.php

```php
<?php
  session_start();
  $_SESSION['name'] = 'Lena';
  $_SESSION['sorte'] = 'Pistazie';
?>
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Seite A</title></head>
<body>
  <p>Gespeichert. <a href="seite_b.php">Weiter zu Seite B</a></p>
</body>
</html>
```

seite_b.php seite_b.php

```php
<?php
  session_start();
?>
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Seite B</title></head>
<body>
  <p>Hallo <?php echo htmlspecialchars($_SESSION['name'] ?? 'Gast'); ?>,
     Ihre Lieblingssorte ist <?php echo htmlspecialchars($_SESSION['sorte'] ?? 'unbekannt'); ?>.</p>
</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS



Abbildung 2: Seite B kennt die Werte, die Seite A gespeichert hat.

3 | Ganz am Anfang!

Das Session-Cookie wird im **Kopf** der HTTP-Antwort übertragen – vor dem eigentlichen HTML. Hat PHP schon etwas ausgegeben, ist der Kopf bereits abgeschickt:

Fehler: Ausgabe vor session_start() zu_spaet.php

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Zu spät</title></head>
<body>

<p>Willkommen!</p>
<?php
  session_start();
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

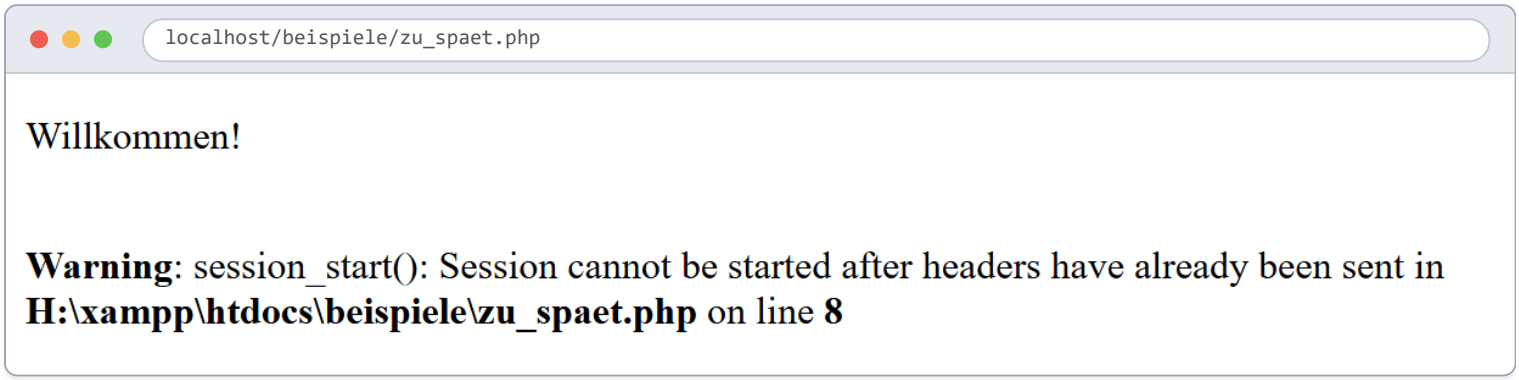


Abbildung 3: session_start() nach einer Ausgabe – so sieht es auf einem Server ohne Ausgabepuffer aus.

⚠ VOR JEDER AUSGABE

`session_start()` und `header()` müssen stehen, bevor irgendetwas ausgegeben wird – auch kein Leerzeichen und keine Leerzeile vor `<?php`. Deshalb steht `require 'schutz.php';` in der allerersten Zeile.

💡 WARUM ES BEI XAMPP MANCHMAL TROTZDEM KLAPPT

XAMPP sammelt die ersten 4 KB einer Seite in einem Puffer (`output_buffering=4096` in der `php.ini`), bevor es sie abschickt. Bei kurzen Seiten geht ein zu spätes `session_start()` deshalb scheinbar gut. Auf Servern ohne diesen Puffer oder bei längeren Seiten erscheint die Warnung aus Abbildung 3 – und die Anmeldung funktioniert nicht. Verlassen Sie sich nie auf den Puffer.

4 | Weiterleiten mit header()

Weiterleitung

```
header('Location: login.php');   // Browser soll login.php aufrufen
exit;                            // und hier ist Schluss!
```

`header()` schickt dem Browser die Anweisung, eine andere Seite zu laden. PHP arbeitet danach aber **weiter**! Ohne `exit` würde der Rest der geschützten Seite trotzdem erzeugt und mitgeschickt – ein Angreifer, dessen Programm die Weiterleitung ignoriert, bekäme alle Daten.
